

AA-HW04

Maria Camila Castaño Martínez

# Analysis of Algorithms

## Homework 4

1. On notebook (.ipynb)

2. 12 coins  $\rightarrow$  coin on 3 weightings

\* First, The initial split

↳ Division of the 12 coins into 3 groups of 4:

$\{1, 2, 3, 4\}, \{5, 6, 7, 8\}, \{9, 10, 11, 12\}$

$\rightarrow$  Assuming weighing 1:

For now, placing  $\{1, 2, 3, 4\}$  on

the left scale and  $\{5, 6, 7, 8\}$  on

the right scale

\* Second step, let's branch the logic

↳ Now, let's present 2 possible cases:

\* Case 1: Balanced scale

if the scale is equal, the odd coin is in the group  $\{9, 10, 11, 12\}$ , meaning the coins 1-8 are even



Weighing 2 of  $\{1, 2, 3\}$  against  $\{9, 10, 11\}$  for testing to expect 1 of 3 possible outcomes:

- Balanced: 12 is the odd coin



W3: 12 against 1 (✓),

if 12 is lower  $\rightarrow$  heavy

if is higher  $\rightarrow$  light

- 9, 10, 11 is heavier: The odd coin is one of these 3 and it's heavy



W3': 9 vs 10 → if they balanced.  
11 is the heavy  
coin

- 9, 10, 11 is lighter: The odd coin is one of these 3 and it's light



W3'': 9 vs 10 → if they balanced.  
11 is the light  
coin

\* Case 2: Moved scale (either side)  
Meaning either one of {1, 2, 3, 4} is  
Heavy (H) or one of {5, 6, 7, 8} is  
light (L)



Weighing 2: "Swap and drop"

We move some coins, swap others, and bring in "good" coins.

Weigh  $\{1, 2, 5\}$  (2H, 1L) against  $\{3, 6, 9\}$  (1H, 1L, 1 good). Now we test for 1 of 3 possible

outcomes:

- Balanced: The odd coin is out,  $\{4(H)\}$  or  $\{7(L), 8(L)\}$



W3: Weigh 7 vs 8, the lighter one is the odd coin. If they balanced, 4 is the heavy coin

- Left stays heavier: 1 or 2 is heavy, or 6 is light (same orientation)



W3': 1 vs 2. The heavier one is the odd coin, if they balance, 6 is the odd one

- Scale tips the other way:

Either 3 is heavy or 5 is light (swapped coins)



W3'': 3 vs 9 (good). If 3 is heavier, it is the odd coin. If there's a balance, 5 is the light coin

3. The most efficient way to solve this problem is a Staircase Search Algorithm to sort rows and columns



Let's start at the top-left corner  $(0,0)$ .

For its logic:

- Current element  $(0)$ : One found.

Rows are increasing and columns are decreasing. There can be no other zeros in this row or column. Hence, we increment our counting and move diagonally to  $(row+1, col+1)$

- Currently is  $>0$ : Rows increase to the right, every other element in this row must also be  $>0$ . Then, we can safely eliminate the rest of this row and

move down TO the next row:  $row + 1$

- Current is  $< 0$ : When columns decrease as we go down, every other element in this column must also be  $< 0$ . Then we can safely eliminate this column and move to the right:  $col + 1$

### Complexity Analysis:

- Time Complexity:  $O(n)$ ,  
on the while loop, we can increment  $row$  or  $column$ , or both



Max value  $row$  or  $col$  ( $n$ )

↳ Runs at most  $2n$  times

- Space Complexity:  $O(1)$ ,  
use of few ints ( $row$ ,  $col$ ,  $count$ )

↳  $(i, j) = (1, n)$

- if  $A[i][j] = 0 \rightarrow$  Move diagonally:

$i++, j--$

- if  $A[i][j] > 0 \rightarrow$  Move left ( $\downarrow$ )

$j--$

- if  $A[i][j] < 0 \rightarrow$  Move down ( $\uparrow$ )

$i++$

STOP when:

$i > n$

$j < 1$

4.  $A[1 \dots n] \rightarrow$  From 1 to  $n^2$

Code can be found on the notebook

Since at most  $\log \log n$  distinct elements appear in  $A$ , we first compute the frequency of each element using a hash table in  $O(n)$  time.

We then sort the distinct keys in  $O((\log \log n) \log \log \log n)$  time and reconstruct the sorted array in  $O(n)$  time

$$\hookrightarrow O(n) < O(n \log n)$$

## 5. Consistent 9-TO-1 split

a) Recurrence relation.

If the algorithm can split the input of size  $n$  into subproblems of size

$$\frac{9}{10}n \text{ and } \frac{1}{10}n$$

$$\hookrightarrow T(n) = \underbrace{T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right)}_{\text{Times To Solve The Subproblems}} + \overset{\substack{\text{Time Taken} \\ \text{For Partition}}}{c n}$$

Times To Solve  
The Subproblems

## b) Recursion Tree Analysis

↳ On level 0 (roots) =  $Cn$

↳ On level 1, the total work at

this level:

$$c \left( \frac{9}{10} n \right) + c \left( \frac{1}{10} n \right) = Cn$$

↳ On level 2, each problem will split again proportionally

↳ At every level, the total size of subproblems still adds up to  $n$ .

c) Height of the Tree, by determining the largest branch:

$$n \rightarrow \frac{9}{10} n \rightarrow \left( \frac{9}{10} \right)^2 n \rightarrow \dots$$

Will stop when:

$$\left(\frac{9}{10}\right)^k n = 1$$

Solve for  $k$ :

$$\left(\frac{9}{10}\right)^k = \frac{1}{n}$$

Take logs:

$$k \log \left(\frac{9}{10}\right) = -\log n$$

$$k = \Theta(\log n)$$

d) Total work:

For each level =  $Cn$

Number of levels =  $\Theta(\log n)$ ,

→ So Total cost:

$$T(n) = (n \cdot \Theta(\log n))$$

$$T(n) = \Theta(n \log n)$$

6. Full Python implementation and algorithm analysis is in the notebook "HWADA\_4"